



01211373 Machine Learning and Programming for Industry

A Tour of Classifiers & Data Preprocessing

Lecture 2 — Classical Machine Learning

Last Week → This Week

WEEK 1 RECAP

- ML complements classical control
- Supervised / unsupervised / reinforcement learning
- The build-a-model roadmap: preprocess → train → evaluate → deploy
- scikit-learn now, PyTorch from Week 7

TODAY

- Four classifiers: logistic regression, k-NN, SVM, decision trees
- Why raw sensor data usually isn't model-ready
- Handling missing values & categorical fields
- Feature scaling and why some models need it
- Chaining it all into a scikit-learn Pipeline

Today's Running Example: Multiclass Fault Detection

Week 1 was binary (normal vs. faulty). Real machines fail in more than one way — today we classify among four operating states.

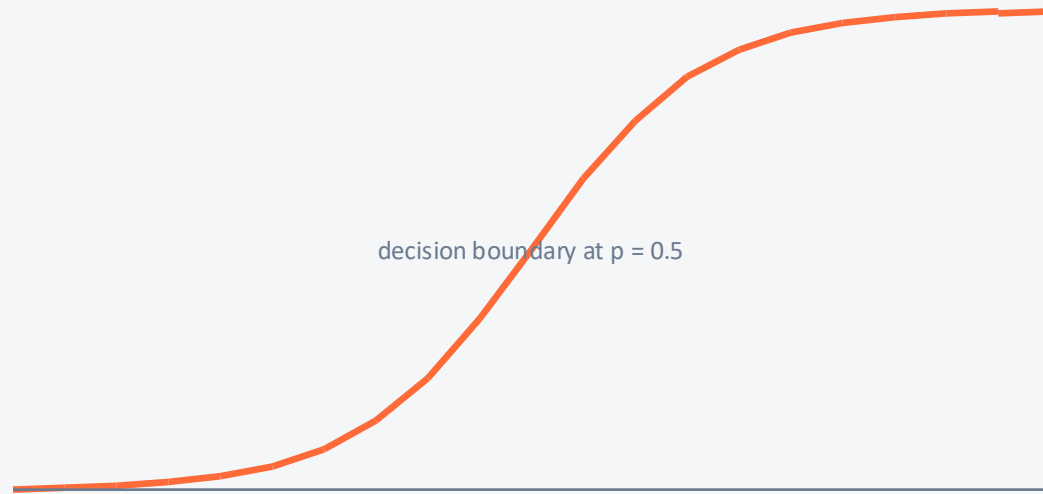
Normal	Bearing Fault	Misalignment	Imbalance
Smooth periodic vibration, no anomalies	Sharp periodic impulses from impact events	Elevated 2× running-speed harmonic	Elevated 1× running-speed amplitude

Every classifier and preprocessing technique today will be demonstrated on this four-class problem, and you'll build it yourself in the lab.

Logistic Regression



A linear model that squashes its output through a sigmoid to produce a class probability.



Robotics use

A fast, interpretable baseline — fit it first before reaching for anything fancier. Coefficients tell you which sensor features drive the fault call.

Needs scaled features

k-Nearest Neighbors (k-NN)



Classify a new point by majority vote among its k closest training examples — no training phase, just distance lookups.

Strengths

- Simple, no training step
- Naturally handles multiclass
- Nonlinear decision boundary

Watch out for

- Slow to predict on large datasets
- Sensitive to feature scale
- Curse of dimensionality

Support Vector Machines (SVM)



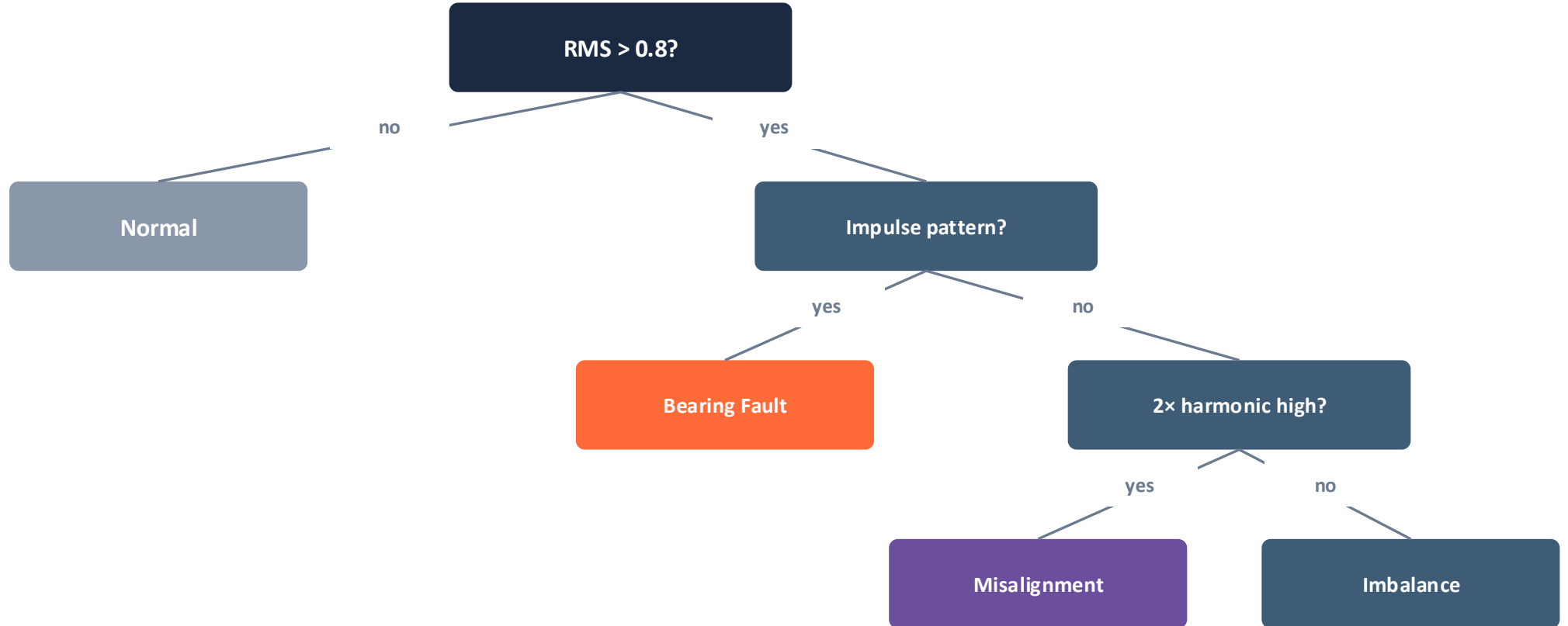
Finds the decision boundary that maximizes the margin — the gap — between classes; kernels let it bend that boundary for nonlinear data.

Robotics use: strong when you have a clear-margin, moderate-size dataset — e.g., separating a handful of well-characterized fault types from clean vibration features.

Needs scaled features, and doesn't scale gracefully to very large datasets — training time grows quickly with the number of examples.

Decision Trees

Learns a sequence of if/else splits on individual features — the only classifier here that doesn't need scaled inputs.



Choosing a Classifier

	Logistic Reg.	k-NN	SVM	Decision Tree
Interpretable?	Yes	Somewhat	No (kernel)	Yes
Needs scaling?	Yes	Yes	Yes	No
Nonlinear boundary?	No	Yes	With kernel	Yes
Fast to train?	Yes	Instant*	Slower at scale	Yes

**Training is instant because k-NN just stores the data; the real cost shows up later, at prediction time.*

Why Data Preprocessing Matters



Real sensor logs are rarely model-ready. Every classifier from the last four slides will silently underperform — or outright fail — on data like this.

Missing readings

A sensor dropout leaves a NaN mid-log

Categorical fields

Machine type, shift, operator ID aren't numbers

Mismatched scales

Temperature in °C next to vibration in mm/s

Handling Missing Data



scikit-learn models won't accept NaNs — you have to decide, deliberately, what to do with each gap.

Drop

Remove rows/columns with missing values. Simple, but wastes data — fine only if gaps are rare.

Impute

Fill with the column mean/median (or a smarter model). The default choice for sparse, random gaps.

Flag & Fill

Impute, but add a binary “was_missing” column so the model can learn from the gap itself.

Categorical Data → One-Hot Encoding



Models need numbers. A column like `machine_type` can't be fed in as text — and can't just be numbered 0, 1, 2 either.

machine_type

CNC

Press

Lathe



is_CNC

1

0

0

is_Press

0

1

0

is_Lathe

0

0

1

One column per category, each 0/1. In scikit-learn: `OneHotEncoder()` inside a `ColumnTransformer`, or `pd.get_dummies()` for a quick look.

Feature Scaling



Logistic regression, k-NN, and SVM all compare distances or magnitudes across features — an unscaled feature can silently dominate.

StandardScaler

Rescales to mean 0, std 1. The default choice for most sensor features.

MinMaxScaler

Rescales to a fixed [0, 1] range. Useful when you need bounded inputs.

Golden rule: fit the scaler on the training set only, then use it to transform both train and test — never fit on test data.

Putting It Together: scikit-learn Pipelines



A Pipeline chains preprocessing and the model into one object — so “fit on train only” happens automatically, every time.



```
pipe = Pipeline([('preprocess', column_transformer), ('clf', LogisticRegression())])
```

Today's Lab: Preprocess, Then Compare Four Classifiers

1

Load a messy multiclass dataset

Missing values, a categorical column, four fault classes.

2

Clean & encode

Impute missing readings; one-hot encode the categorical field.

3

Build a Pipeline

Chain preprocessing and scaling with ColumnTransformer.

4

Compare classifiers

Logistic regression, k-NN, SVM, decision tree — same data, same split.

Key Takeaways



Four classifiers, one interface: `fit()`, `predict()`, `score()`.



Trees skip scaling; logistic regression, k-NN, and SVM all need it.



Pipelines make “fit on train only” automatic — use them by default from now on.

NEXT WEEK

Regression Analysis

Predicting continuous quantities — sensor calibration and system identification.